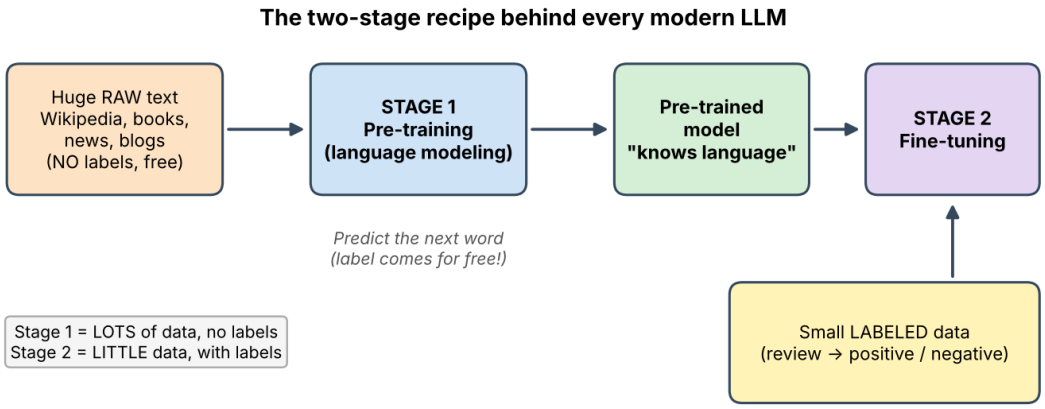
Week 2 Study Notes

Language Modelling, Pre-training, Fine-tuning & the GPT Architecture



Notes follow the order of your syllabus: L12 → L11 → L13 → L14 → L16 → L15.

Chapter	Video	What you will understand
1	L12	Why we need pre-training (and why labels are a problem)
2	L11	What a language model is; why predicting the next word = learning language
3	L13	How Transformers plug in: encoder-only, decoder-only, encoder-decoder
4	L14	Decoder-only models: the mask, autoregressive generation, the training objective
5	L16	GPT-1 in detail: data, sizes, one block step by step, counting parameters
6	L15	Pre-training vs fine-tuning, task formats, text generation and its problem

Intuition: How to use these notes: every chapter starts with a one-line goal, uses a small example, and has a picture. If a section feels heavy, look at the picture first, then read the text. Yellow boxes = things to memorise.

CHAPTER 1 · VIDEO L12

Why Pre-training? The Motivation

In one line: Labeled data is scarce and costly, raw text is everywhere. Can we use raw text to teach a model language first?

1.1 The starting point: Transformers as a general function

In the previous week you saw the Transformer for machine translation (English → Hindi). Zoom out and ignore the details (embeddings, attention, feed-forward, residuals, normalisation): it is just a **function with parameters θ** that maps an input text to an output text. Many NLP tasks fit this same shape:

Task	Input	Output
Translation	sentence in language A	sentence in language B
Sentiment analysis	a review	class: positive / negative / neutral
Summarisation	long paragraph	1–2 line summary
Question answering	paragraph + question	the answer span (token with highest probability)

θ stands for all the parameters: W_Q, W_K, W_V, feed-forward weights, repeated across every layer.

1.2 The problem: labeled data is the bottleneck

- To train for each task you need **labeled** data (input + correct output).
- Training **from scratch** means randomly initialising a huge number of parameters, so it needs lots of data and time to converge.
- Labels are expensive: you must hire skilled human annotators, and it takes time (especially for translation, QA, summarisation).
- For languages like Hindi there are very few QA or sentiment datasets.

1.3 The opportunity: unlabeled data is free and huge

Wikipedia, news websites, books (Wikibooks, Internet Archive), blogs: all of this is just text. It is called **unlabeled** because nothing says “this paragraph is positive” or “here is a question and its answer”.

> **Remember:** Two questions this week answers: (1) What training objective can we use if there are no labels? (2) Does learning from raw text really reduce the labeled data we need later?

1.4 Human analogy (the core intuition)

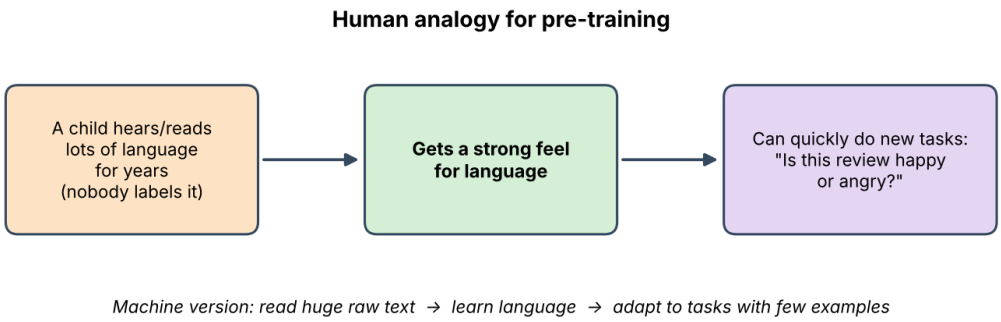


Figure 1.1: Humans learn language from exposure, not from task-specific lessons.

Ask an average person: *"Wow, India has reached the Moon!"* is that positive or negative? They answer instantly. Nobody trained them on "sentiment analysis". Over a lifetime of talking, reading, watching movies they absorbed language: angry words appear when people shout, appreciative words when people praise. Likewise they can read a paragraph and answer "did the lander use AI?", or tell that *"he likes to sway"* is a sensible sentence.

Example: Imagine teaching sentiment analysis to someone who knows **no language**: you would first have to explain what words are, which are good, which are bad. Painful. If they already know the language, a few examples are enough. That is the gap between a model with and without pre-training.

1.5 The two-stage recipe

- **Stage 1 – Pre-training:** expose the model to huge raw text (this is "language modeling", next chapter).
- **Stage 2 – Supervised fine-tuning:** adapt the model to a task with a *small* labeled dataset.

Result shown in the lecture (sentiment analysis): the two-stage model reached about **92% accuracy**, beating the then state of the art, and matched older purely-supervised systems using **30× to 100× fewer labels**. (Numbers are from the time of those papers; modern systems are better.)

Watch out: "Few" labels is relative: it means far fewer than a model starting with zero knowledge of language would need.

CHAPTER 2 · VIDEO L11

Language Modelling: Pre-training Made Mathematical

In one line: A language model assigns probabilities to word sequences; training it to predict the next word turns free raw text into free labeled data.

2.1 Notation

- **V** = vocabulary: the set of all unique words of the language.
- A sentence is a sequence $\mathbf{x_1, x_2, \ldots, x_n}$, each $x_i \in V$, and n can be any length.

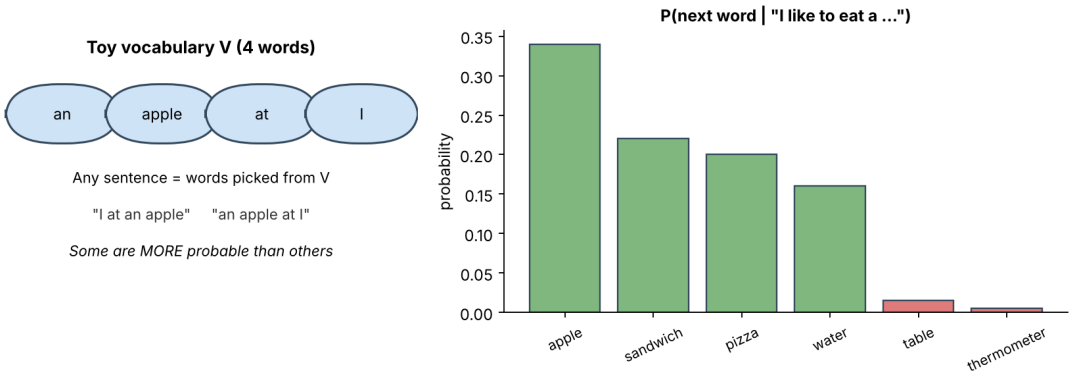


Figure 2.1: Left: a toy 4-word vocabulary. Right: some next words are far more probable than others.

Example: Toy language with V = {an, apple, at, I}. Possible sentences: “an apple”, “I at an apple”, “at I I”. In real English, “I want a glass of water” is far more likely to appear in a huge corpus than a sentence about a very rare medical condition. **Some sentences are more probable than others.**

2.2 What is a language model?

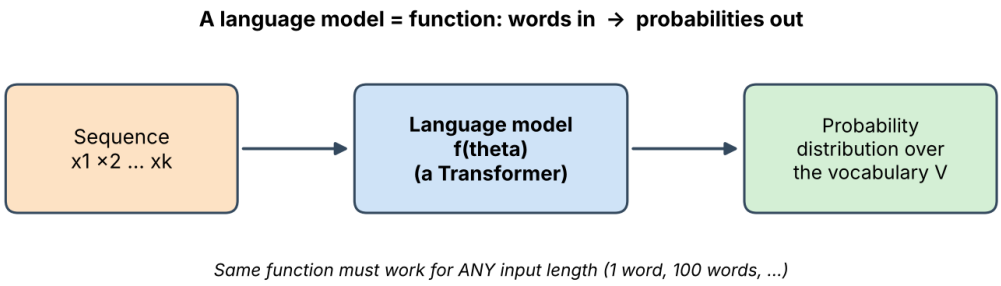


Figure 2.2: A language model is a function from a sequence to probabilities.

We want a function that takes a sequence (any length) and gives a **probability** for it. You have seen such functions: every deep network takes an input and outputs (often) a probability distribution. A function like this is called a **language model**.

2.3 The chain rule

Using the chain rule of probability, the probability of a whole sequence is a product of conditional probabilities:

```
P(x1, x2, ..., xT) = P(x1) × P(x2 | x1) × P(x3 | x1,x2) × ... × P(xT | x1..xT-1)
```

- Each factor is: "probability of the next word given **everything before it**".
- (The lecturer notes the product should start at i = 2 with $P(x_1)$ outside, since x_0 doesn't exist.)
- **Naive shortcut:** assume words are independent, so $P = P(x_1)\cdot P(x_2)\cdot P(x_3)\ldots$ This is wrong in practice, so we **don't** use it.

Example: "I enjoyed reading a **book**" is much more likely than "I enjoyed reading a **thermometer**" because "enjoyed reading" makes "book" likely. Words depend on earlier words, so independence fails.

2.4 Closing the loop: pre-training = language modeling

"Understand language" is vague. We make it concrete with a **simplifying assumption**:

Remember: If a model can predict the next word well, it must have understood the words so far. So: **teach the model to predict the next token**.

Honesty note from the lecture: this is not how humans learn (parents don't make us predict the next word), and it is not proven that good next-word prediction means real understanding (some researchers disagree). It is simply the working assumption that motivates pre-training.

2.5 The magic trick: labels for free

Take any sentence from the web. Give the first k words as input X and the (k+1)-th word as the target y. Someone already wrote the sentence, so the "label" costs nothing. One sentence of length L gives about L−1 training samples, with k from 1 up to L−1.

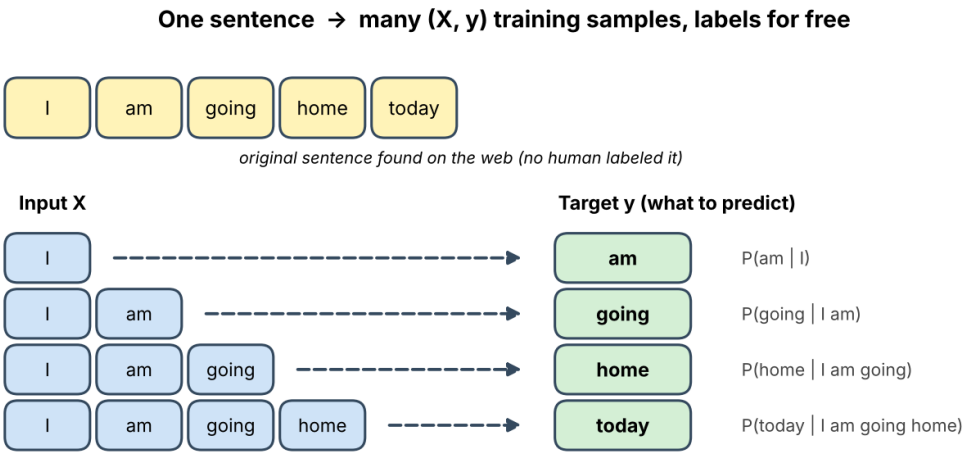


Figure 2.3: One unlabeled sentence becomes many supervised (X, y) pairs.

Asking the model to predict y given X means: **output a probability distribution over the whole vocabulary** and the correct word should get the highest probability. This is exactly the kind of output (softmax over vocabulary) that Transformers already produce.

2.6 What exactly do we learn?

- Many conditional distributions: $P(x_1)$, $P(x_2|x_1)$, $P(x_3|x_1,x_2)$, ...
- They look different because the *condition* changes, but the thing being predicted is always the same kind of object: **a distribution over V**.
- So one model with one set of parameters can be reused at every position.

> **Intuition:** Later models (e.g. BERT) use a variant: predict *masked* tokens given others. The abstraction is the same: **given some tokens, predict some token**.

CHAPTER 3 · VIDEO L13

Using Transformers for Language Modelling

In one line: We need one flexible function F(θ): any-length input → distribution over V. A Transformer fits. There are three ways to build it.

3.1 The learning problem

Given input $x_1 \ldots x_k$ we want $P(x_{k+1} \mid x_1 \ldots x_k)$. We need **one** function that works for any length (not a separate function for 2-word inputs, another for 4-word inputs). The Transformer can do this. Our job: learn θ so the right next word gets the highest probability.

3.2 Autoregressive and causal language modeling

- **Autoregressive model:** parameterised function (fixed number of parameters θ) that predicts each conditional probability from earlier words. Transformers, RNNs and feed-forward nets can all play this role.
- **Causal language modeling:** given the words so far (the “cause”), predict the next word.
- We use the network for each *term* of the chain-rule product (conditional probabilities). Multiply them later if you ever need the whole-sentence probability.

3.3 Three Transformer families

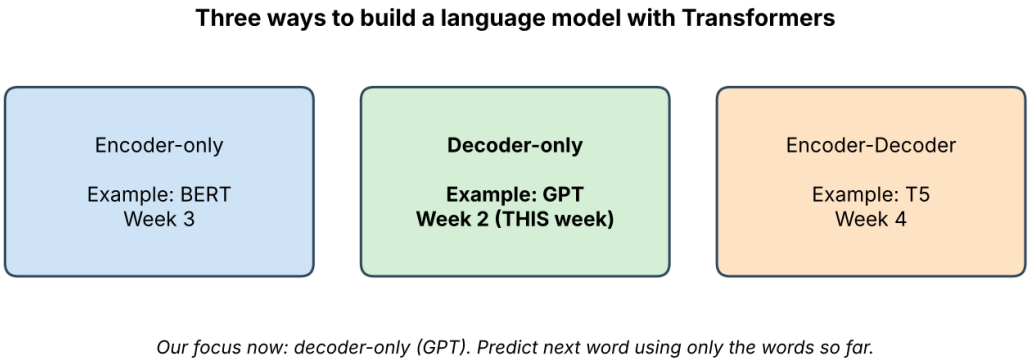


Figure 3.1: The course roadmap: one family per week.

Family	Meaning	Model in this course	Week
Encoder-only	Only the encoder half of the Transformer	BERT	3
Decoder-only	Only the decoder half	GPT	2 (now)
Encoder-decoder	Both halves (like the translation Transformer)	T5	4

> **Remember:** Almost every modern LLM maps to one of these three families, with small differences. That is why this course is organised around them.

CHAPTER 4 · VIDEO L14

Causal LM: Decoder-only Transformers

In one line: Take the decoder, delete cross-attention, keep masked self-attention: it predicts the next word from the past only.

4.1 From the translation decoder to a decoder-only block

In translation, the decoder has masked self-attention, **cross-attention** (looking at the encoder's output) and a feed-forward network. For language modeling there is **no encoder** and nothing to cross-attend to, so cross-attention is removed.

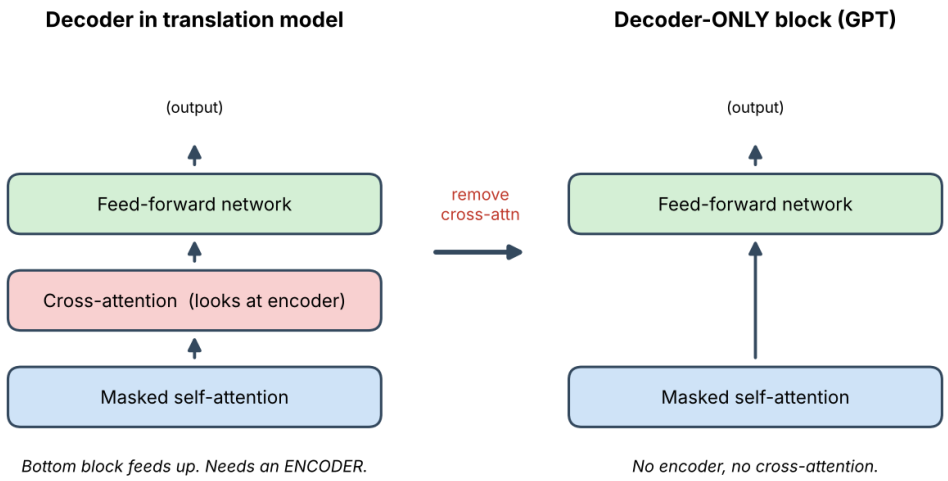


Figure 4.1: Decoder-only block = masked self-attention + feed-forward. Nothing else.

4.2 Why we must hide the future (the mask)

During training we have the whole paragraph, but when the model is used, the user gives only a prompt such as *"I am going to"*. If the model could peek at later words while predicting the 5th word, the task would be trivially easy and useless in the real world. So the model may look only at the **present and past**.

Self-attention computes a new vector for word j as a weighted sum of value vectors: $h_j = \Sigma_i \alpha_{ji} \cdot v_i$. The mask forces $\alpha_{ji} = 0$ for every future word $i > j$.

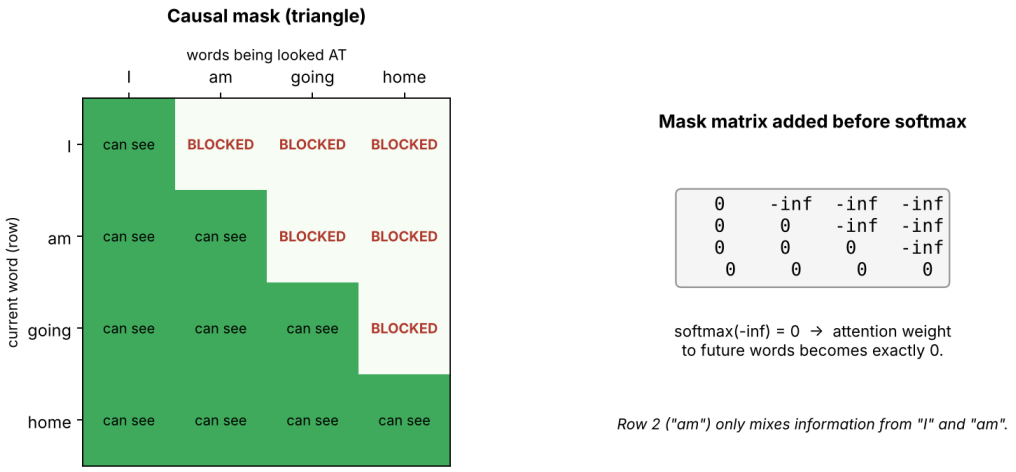


Figure 4.2: Causal mask. Add $-\infty$ to the scores of blocked positions; softmax turns them into exactly 0.

Example: Sentence “I am going home”. When computing the representation of “am”, it can use “I” and “am”. It **cannot** use “going” or “home”. “home” can use all four words.

4.3 Autoregressive generation

The model first predicts a distribution for word 1 ($P(x_1)$, not conditioned on anything), then feeds the output back as input to predict word 2, and so on. To predict x_4 it uses x_1, x_2, x_3, giving $P(x_4 \mid x_3, x_2, x_1)$: exactly the chain rule. At each step the output is a softmax over the vocabulary computed by the Transformer from its parameters θ (including the word embeddings, which are part of θ).

4.4 Training objective

- Maximise the **likelihood** $L(\theta)$ of the real training text.
- For the training sentence “I am going home today”: when the input is “I”, the probability of “am” should be maximal; when the input is “I am”, the probability of “going” should be maximal; and so on.
- Use ordinary **back-propagation** (from the first course) to adjust θ until the correct word consistently gets the highest probability.

Remember: Training loop in plain words: show text → model guesses next-word probabilities → compare with the real next word → adjust θ → repeat on billions of words.

CHAPTER 5 · VIDEO L16

GPT-1: Architecture & Parameter Count

In one line: Stack 12 decoder-only blocks, train on continuous 512-token chunks of books, predict every next token in parallel.

5.1 Stacking blocks

GPT (Generative Pre-trained Transformer) is the first decoder-only model of the GPT family (the lecture mentions GPT-4 as the latest at that time). Stack N identical blocks. Each layer produces a refined representation of every input token; the last layer goes through a linear layer and softmax to give a vocabulary distribution.

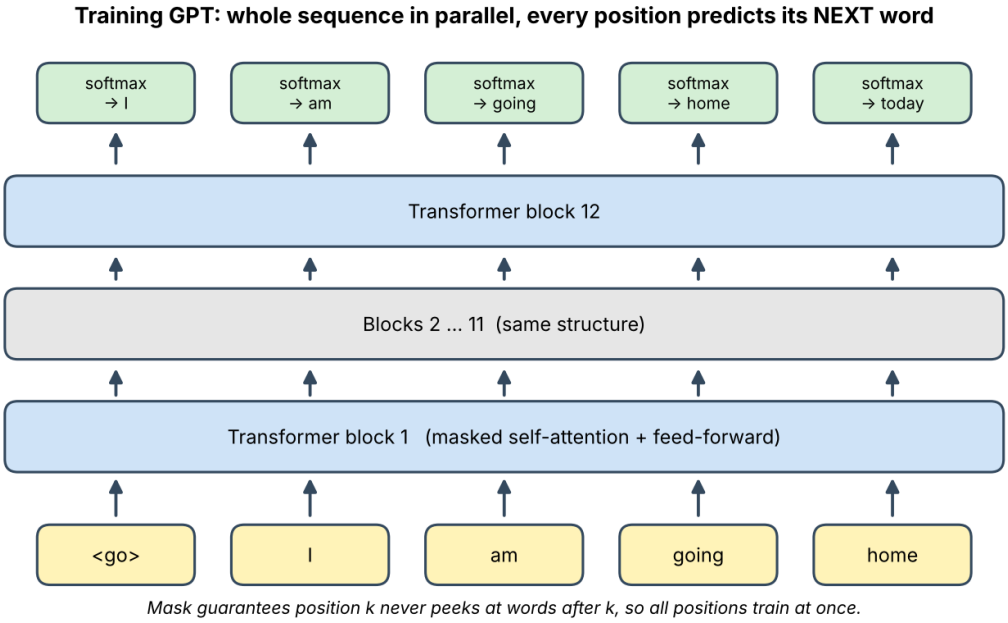


Figure 5.1: The input is the text shifted by one: at position k the target is word k+1. The mask lets all positions train at the same time.

- Layer 0 = word embeddings: a T × d_{model} matrix (T tokens, each a vector of size d_{model}).
- Block ℓ takes the output of block $\ell-1$; the first block takes the raw embeddings.
- Loss = sum of the log-probabilities of the correct next word at all T positions (maximise the sum, or equivalently minimise the negative log likelihood).
- Training in parallel is possible **only because of the triangular mask**.

5.2 Training data

- **BookCorpus:** about 7,000 unique books, 74 million sentences, about 1 billion words. Tiny compared with modern LLMs.
- **Long contiguous text:** each training sequence is 512 consecutive tokens from a book (sentences are not shuffled inside it).
- **Byte-pair encoding (BPE) tokenizer:** covered later with BERT. Mentally treat one space-separated word as one token for now. Real tokenizers can split frequent suffixes like “ing” (e.g. “going” → “go” + “ing”).

5.3 Model size at a glance

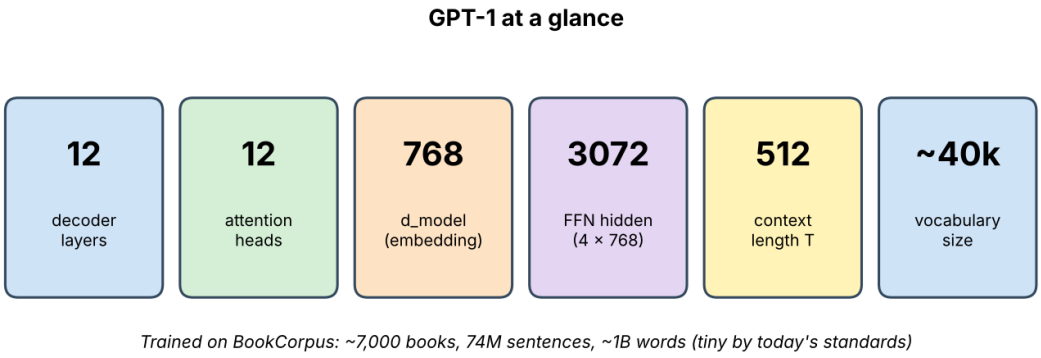


Figure 5.2: GPT-1 hyper-parameters.

Quantity	Value	Meaning
Layers	12	12 decoder blocks in a row
Context size T	512	Model sees 512 tokens at once
d_{model}	768	Embedding size; also the size at the input and output of every block
Attention heads	12	Each head outputs 768/12 = 64 dimensions
FFN hidden size	3072	4 × 768 (an empirical choice, no theory behind it)
Activation	GELU	Used only in the feed-forward layer
Vocabulary	≈ 40,000	Number of distinct tokens in the softmax output
Regularisation	Dropout, LayerNorm, residuals	Faster, stabler training

5.4 Multi-head attention with real numbers

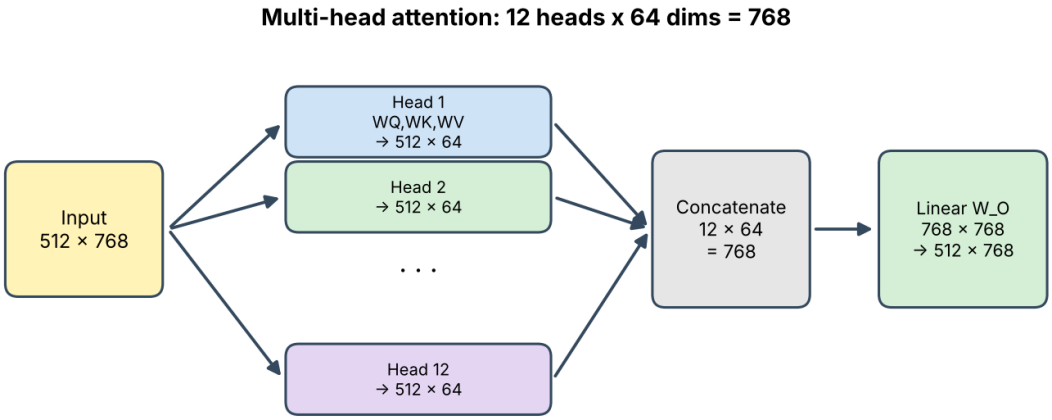


Figure 5.3: 12 heads × 64 = 768, then W_O mixes the heads.

- Input: 512 × 768. Every head has its **own** W_Q, W_K, W_V (each 768 × 64) and computes Q, K, V for all 512 tokens.
- Each head runs the usual masked attention and produces 512 × 64.
- Concatenate 12 heads → 512 × 768. Multiply by W_O (768 × 768) → 512 × 768.

Remember: Golden rule: number of inputs = number of outputs at every layer, and d_{model} stays constant (768) through the whole network.

5.5 One block, step by step

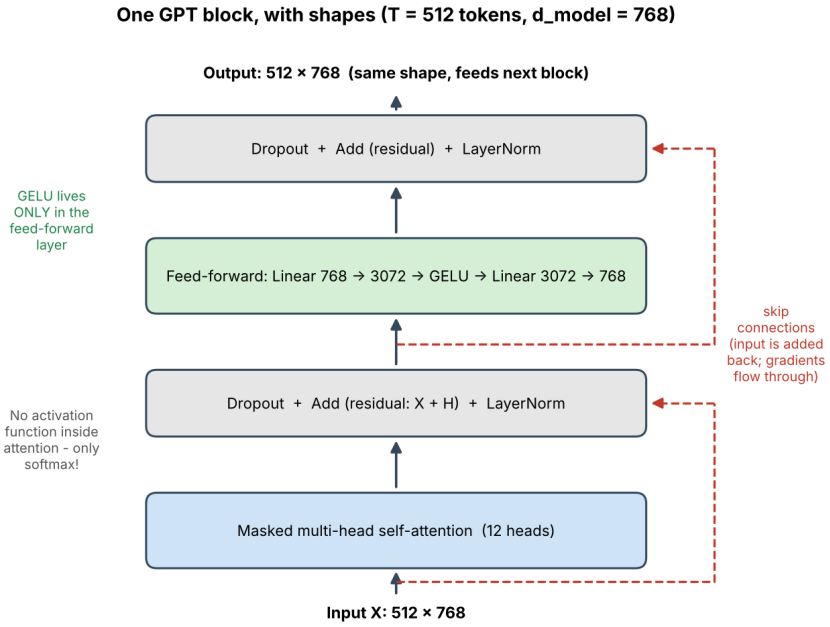


Figure 5.4: One GPT block (shapes are for a single sequence of 512 tokens).

- **Step 1:** masked multi-head attention turns X (512×768) into H (512×768).
- **Step 2:** Dropout, then residual add X + H = H′, then LayerNorm. The skip path lets gradients flow straight back, as in ResNets, which matters for deep networks.
- **Step 3:** feed-forward, applied to each of the 512 positions independently: 768 → 3072 → (GELU) → 768.
- **Step 4:** Dropout, residual add (input of the FFN + its output), LayerNorm. Output 512×768 goes to the next block.

Watch out: Attention has **no activation function**, only the softmax that creates attention weights. The only activation (GELU) sits inside the feed-forward layer.

5.6 Counting the parameters

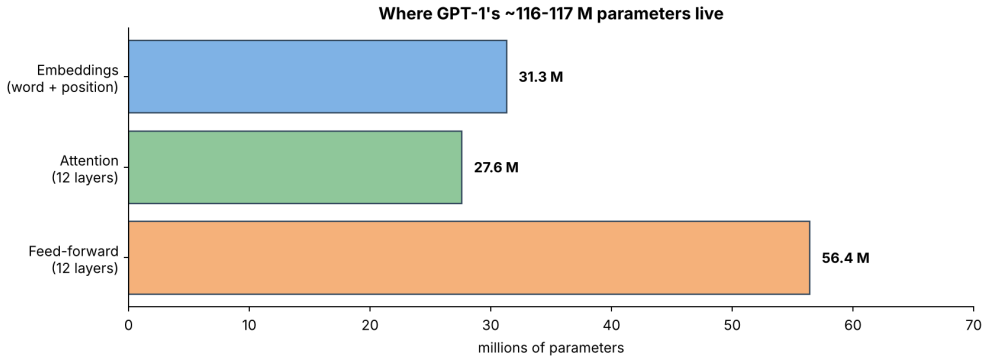


Figure 5.5: Parameter budget (numbers as given in the lecture).

Part	How it is computed	Result
Word embedding	V × d = ≈40,000 × 768	≈ 31 M
Position embedding	512 × 768 (learned here, unlike the sinusoidal ones in the original Transformer)	≈ 0.4 M
Embeddings total		≈ 31.3 M
Attention per layer	Q,K,V: 3 × (768×64) × 12 heads ≈ 1.7 M; W_O: 768×768 ≈ 0.6 M	≈ 2.3 M
Attention, 12 layers	2.3 M × 12	≈ 27.6 M
FFN per layer	768×3072 + 3072×768 + biases (3072 + 768)	≈ 4.7 M
FFN, 12 layers	4.7 M × 12	≈ 56.4 M
Total	31.3 + 27.6 + 56.4	**≈ 115–117 M**

Intuition: The final 768 → 40,000 output matrix is **not counted again** because it can share weights with the input embedding matrix (input = “embedding matrix”, output = “output embedding matrix”). If not shared, extra parameters are added. Official size: about 117 M. Today's models: 7 B, 13 B, 70 B.

CHAPTER 6 · VIDEO L15

Pre-training, Fine-tuning & Text Generation

In one line: Pre-train with next-token prediction, then fine-tune by adding a small head and a task-specific input format. Generation by repeated next-token prediction needs smarter decoding.

6.1 Architecture takeaway

Once you know the Transformer, each new LLM changes little in the blocks. For GPT you only need the numbers (d_{model}, layers, ...), the inputs/outputs and the loss. Improvements like multi-query or grouped-query attention come later as separate topics.

6.2 Pre-training recap

- **Supervision:** implicit; the next token is the label. Unlabeled corpus.
- **Batch size 64** sequences, each 512 tokens, each token 768 dims → input tensor 64 × 512 × 768.
- **Loss:** for every sequence and every position, minimise −log P(correct next token).
- **Optimiser:** Adam with a cosine learning-rate schedule.
- **Teacher forcing:** at every step feed the *true* previous tokens, not the model's own (possibly wrong) predictions. Early in training the weights are near random, so the model's own guesses would be poor and errors would pile up. Teacher forcing gives quicker, stabler convergence.

6.3 Fine-tuning: adapt to a task

Pre-training taught the model to predict next tokens, not to do sentiment, QA or summarisation. Fine-tuning uses **small labeled data** ($x_1 ... x_m$ with label y) and starts from the pre-trained weights. Why “fine-tuning” and not “training”? Because the weights already sit at a good configuration; we only nudge them, instead of starting from random.

Example: Like a person who already reads fluently: show them a few examples of “Is this review positive?” and they pick up the task quickly.

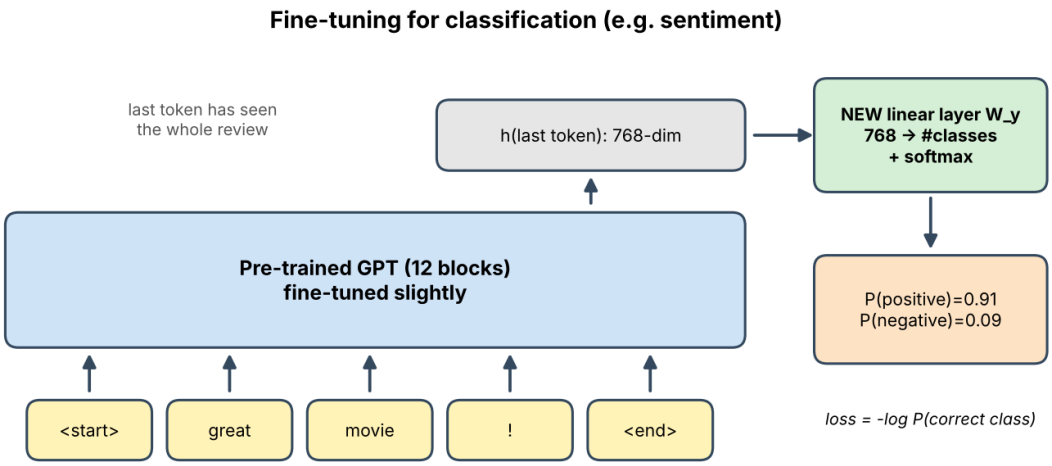


Figure 6.1: Fine-tuning for classification: take the LAST token's final vector, add a new linear layer, softmax over classes.

Why the last token? In GPT's causal setup the last token is the only one that has attended to **all** previous tokens, so its vector summarises the whole review.

- Add a new matrix W_y (768 × #classes), initialised randomly: this is the only new parameter set.
- $P(y \mid x) = \text{softmax}(h_{L,m} \cdot W_y)$, where $h_{L,m}$ is layer L, last token m.
- Loss: $-\log P(\text{correct class})$, summed over all N training reviews (cross-entropy).
- **Sample efficiency:** because of pre-training, fewer labeled examples are needed.
- Sequences shorter than 512 are **padded**; longer ones are **truncated** to 512.

6.4 Freezing choices

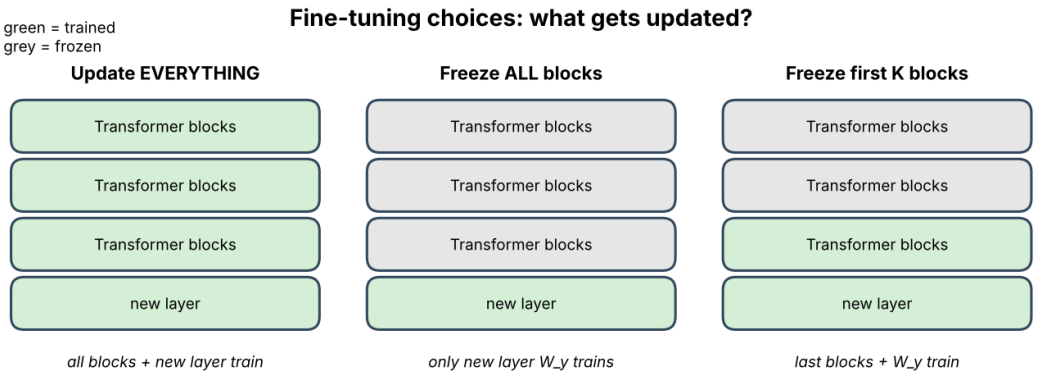


Figure 6.2: You choose how much of the pre-trained network to update.

6.5 Same model, different tasks (only the input format and head change)

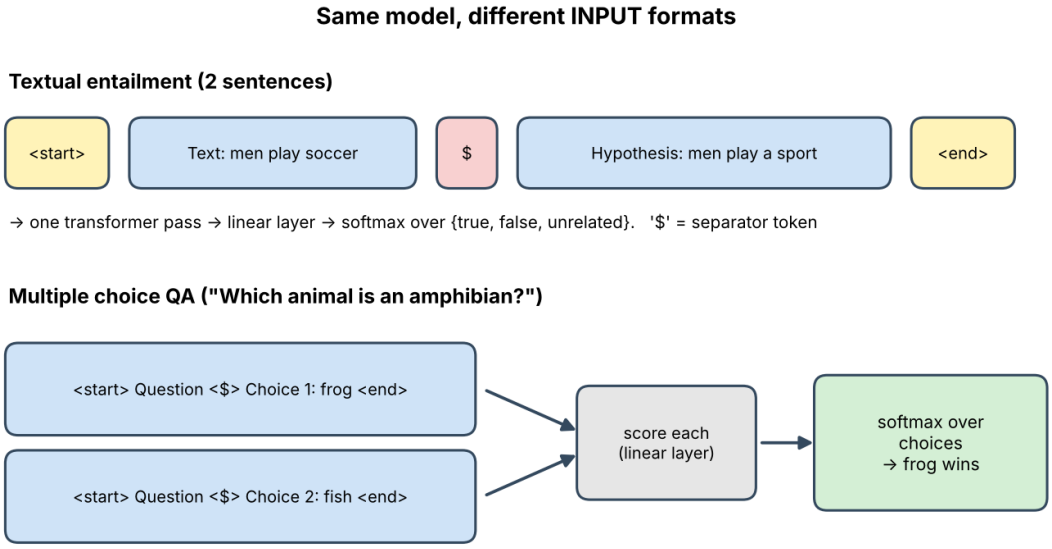


Figure 6.3: Special tokens and the separator ($) let one model handle several task types.

Task	Input format	Output
Sentiment	<start> review <end>	linear layer, softmax over 2+ classes
Entailment	<start> text $ hypothesis <end>	softmax over true / false / unrelated
Multiple-choice QA	One sequence per choice: question $ choice_i	score each choice, softmax across choices

Example for entailment: text “many males are playing a soccer game”, hypothesis “some men are playing a sport” → label **true**.

6.6 Text generation

For generation we ignore the classification head and use the pre-trained next-token output. Prompt “I like” → predict next token → append it → predict again, until a maximum length or an end-of-sequence token.

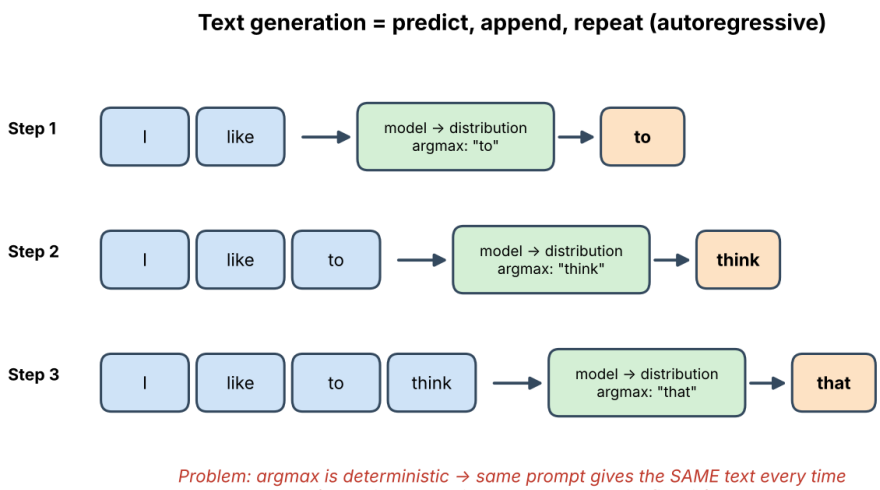


Figure 6.4: Predict, append, repeat.

Watch out: If you always pick the highest-probability token (argmax), the computation is **deterministic**: the same prompt (e.g. “Once upon a time there was a king”) always produces the **identical** story. We want creative, varied output.

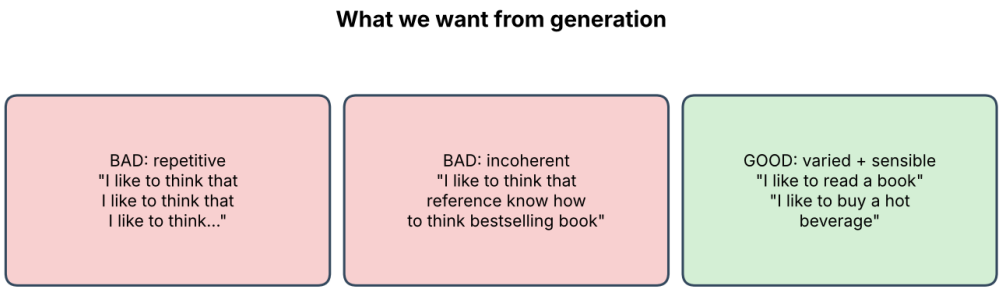


Figure 6.5: Wish list for generated text.

- Not **repetitive** (“I like to think that I like to think that…”).
- Not **incoherent** (random words that don't make sense).
- **Varied**: same prompt can give different, sensible continuations.

The fix is **decoding strategies**, the topic of the next lecture(s); they apply to any large language model, not only GPT.

Revision Sheet

The whole week as a story

- 1. Labeled data is scarce → but raw text is free (L12).
- 2. Language model = function that gives probability of word sequences; chain rule breaks it into “next word given the past” (L11).
- 3. Predict the next word → labels come free from any text → this is **pre-training** (L11).
- 4. A Transformer can be the function; encoder-only / decoder-only / encoder-decoder (L13).
- 5. Decoder-only = masked self-attention + FFN; the mask hides the future (L14).
- 6. GPT-1: 12 layers, 12 heads, d=768, FFN=3072, context 512, ≈117 M parameters (L16).
- 7. Fine-tune by adding a small head + task-specific input; or generate text token by token; argmax decoding is repetitive, so we need better decoding (L15).

Key terms

Term	Simple meaning
Pre-training	Learn language by predicting the next token on huge unlabeled text
Fine-tuning	Adapt the pre-trained model to a task with small labeled data
Language model	Function giving probabilities to sequences / next words
Causal LM	Predict next token using only previous tokens
Autoregressive	Feed each prediction back as input for the next step
Causal / triangular mask	Sets attention to future tokens to 0 via $-\infty$ before softmax
Teacher forcing	Feed the true previous tokens during training
d_{model}	Constant embedding size through the network (768 in GPT-1)
Freezing	Not updating some layers during fine-tuning
Sample efficiency	Needing fewer labeled examples because of pre-training

Check yourself

- Why can a person do sentiment analysis without being trained on it? How does that motivate pre-training?
- Write the chain-rule form of $P(x_1 \ldots x_T)$. Why is the independence assumption wrong?
- How do you turn one sentence of 6 words into training samples? How many?
- What is removed from the translation decoder to get a decoder-only block, and why?
- Why can GPT train all 512 positions in parallel without cheating?
- Where is the activation function in GPT, and which one?
- Why is the last token's vector used for classification?
- Why is argmax decoding a problem for story generation?